Assignment: Advanced Prime Factorization for Cryptography

Our topic is “Determine the Prime Factors of a Given Positive Integer”. In this project, we analyzed the original solution code and found that its running time was higher when processing large numbers, so we wrote a completely new solution to optimize running large numbers. When looking for prime factors, we refer to Pollard’s Rho algorithm. But this algorithm needs another primality check algorithm in order to make it useful, thus, we wrote Miller-Rabin Primality test. By combining these 2 in a main factors function we have implemented a much faster solution.

Problem 1: Implementing a Naive Factorization Algorithm (15 points)
Implement a function **factors_naive** that takes an integer n and returns its prime factors along with their counts using the naive trial division algorithm.

Specifications
1. Start dividing ‘n’ by integers beginning from 2.
2. Record each divisor and its count in the factorization.
3. The function should raise an exception for invalid inputs such as n = 0

Example:
```
# factors_naive (Z.of_int 60);;
- : (Z.t * int) list = [(2, 2); (3, 1); (5, 1)]
```

Problem 2: Pollard’s Rho Algorithm (25 points)
Implement a function **pollards_rho** that takes an integer n and returns a non-trivial factor of n using Pollard’s Rho algorithm. If n is prime, the function should return None.

Specifications
1. Use the g(x) = x^2 + c mod n sequence with a randomized constant c.
2. Detect cycles using Floyd’s cycle-finding algorithm (tortoise and hare).
3. Use lazy evaluation to optimize memory usage.

Example:
```
# pollards_rho (Z.of_string "8051");;
- : Z.t option = Some (Z.of_int 97)
```

Problem 3: Miller-Rabin Primality Test (25 points)
Implement a function **is_probable_prime** that determines if a number n is probably prime using the Miller-Rabin test.

Specifications
1. Input n and a reliability parameter k.
2. Decompose n-1 into 2^s * d, and test k many random bases a
3. Return true if n is likely prime, and false otherwise.

Example:
```
# is_probable_prime (Z.of_int 97) 10;;
- : bool = true
# is_probable_prime (Z.of_int 8051) 10;;
- : bool = false
```

Assignment: Advanced Prime Factorization for Cryptography

Problem 4: Prime Factorization Using Advanced Methods (35 points)
Combine the Pollard's Rho, to find the factors of n, and Miller-Rabin, as a primality test, algorithms into a function, **factors,** that computes the prime factorization of a given integer n.

Specifications
1. Use is_probable_prime to test is n is prime.
2. If n is composite:
   - Use pollards_rho to find a factor.
   - Recursively factorize n and its divisors.
3. Return a list of prime factors and their counts.

Example:
```
# factors (Z.of_int 315);;
- : (Z.t * int) list = [(3, 2); (5, 1); (7, 1)]
```

Extra Credit: Benchmarking and Analysis (10 points)
Benchmark the performance of **factors_naive** and **factors** on a series of integers.
Specifications
1. Create a benchmark for n values ranging from small integers to large numbers.
2. Compare the runtime and scalability of both algorithms.
3. Provide a brief report discussing:
   - At what point does the naive method become infeasible?
   - The efficiency gains achieved by combining Pollard's Rho and Miller-Rabin.

Sources

https://ocaml.org/exercises
https://www.youtube.com/watch?v=7lhlJTtCsiw
https://www.youtube.com/watch?v=zmhUlVck3J0
https://www.geeksforgeeks.org/fundamental-theorem-of-arithmetic/
https://www.geeksforgeeks.org/pollards-rho-algorithm-prime-factorization/
https://www.geeksforgeeks.org/primality-test-set-3-miller-rabin/